SDM120-Zähler-Gateway (D1 Mini ESP32)

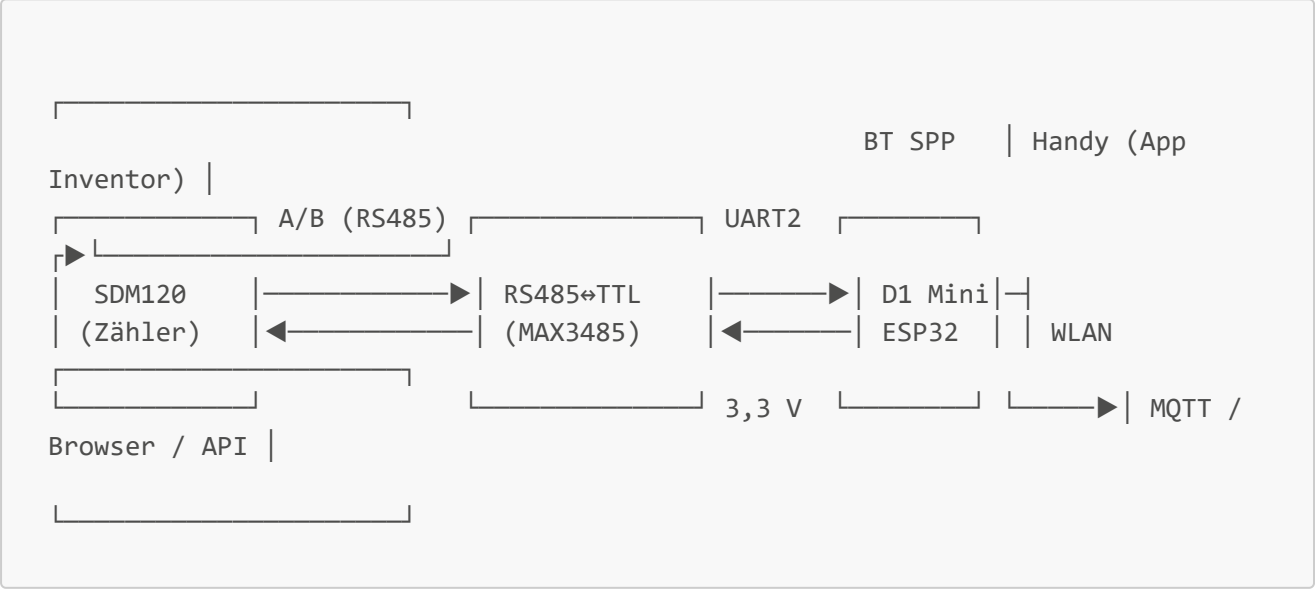
Ein Eastron **SDM120** wird per **Modbus RTU über RS485** von einem **D1 Mini ESP32** ausgelesen. Die Messwerte stehen danach über mehrere Wege zur Verfügung:

- **Bluetooth Classic** für die eigene Handy-App mit MIT App Inventor (Anleitung: `APP_INVENTOR.md`)
- **MQTT** (Einzel-Topics + JSON) für Energiemanagement, Home-Automation und Fernübertragung
- eine **Weboberfläche** unter `http://sdm120.local`
- eine **JSON-API** unter `http://sdm120.local/api`
- Programmieren per **USB-Kabel** aus der Arduino IDE (bewusst ohne OTA-Update über WLAN)

Bluetooth und WLAN laufen **gleichzeitig**. Jeder Teil lässt sich in `config.h` einzeln abschalten.

---

1. Systemübersicht



Der ESP32 ist **Modbus-Master**, der Zähler ist **Slave** (Adresse 1).

---

2. Stückliste

Teil	Bemerkung
Eastron SDM120-Modbus	Version **mit Modbus**. Beim SDM120 gibt es auch eine reine S0-Variante.
D1 Mini ESP32 (ESP32-WROOM-32)	**klassischer ESP32**. ESP32-S3 und ESP32-C3 haben **kein Bluetooth Classic**!
RS485↔TTL-Modul	**für 3,3 V geeignet** (MAX3485 / SP3485), ideal mit automatischer Richtungsumschaltung
Leitung	verdrilltes Adernpaar, z. B. ein Paar aus CAT5/CAT6

Teil	Bemerkung
120-Ω-Widerstand	optional, Busabschluss bei langen Leitungen
5-V-Netzteil	Hutschienennetzteil oder USB-Netzteil, mindestens 500 mA (WLAN und Bluetooth ziehen Stromspitzen)
Android-Handy	Android 6 oder neuer mit Bluetooth

Hinweis zum Pegel

Klassische blaue **MAX485-Module** sind für 5 V gebaut. Laufen sie mit 5 V, liefert der RO-Ausgang 5 V an den ESP32. Der ESP32 ist **nicht 5-V-tolerant**. Ein **MAX3485- oder SP3485-Modul** an 3,3 V ist die saubere Lösung.

---

3. Verdrahtung

D1 Mini ESP32 ↔ RS485-Modul

RS485-Modul	D1 Mini ESP32	Funktion
VCC	3V3	Versorgung 3,3 V
GND	GND	Masse
RO / RXD	IO16	Empfang (UART2 RX)
DI / TXD	IO17	Senden (UART2 TX)
DE + RE (gebrückt)	IO4	Richtung: HIGH = Senden, LOW = Empfangen

- Hat dein Modul eine **automatische Richtungsumschaltung** (keine DE/RE-Pins), setzt du in `config.h` den Wert `PIN_RS485_DE` auf `-1`.
- Beim ESP32 lässt sich der UART auf fast jeden GPIO legen. Andere Pins trägst du einfach in `config.h` ein. **Nicht verwenden**: GPIO 0, 2, 5, 12, 15 (Boot-Pins), 6–11 (Flash), und GPIO 34–39 nur als Eingang (also höchstens für RX).
- Die Beschriftung des D1 Mini ESP32 kann je nach Hersteller abweichen. Maßgeblich ist der GPIO-Aufdruck „IO16", „IO17", „IO4".

RS485-Modul ↔ SDM120

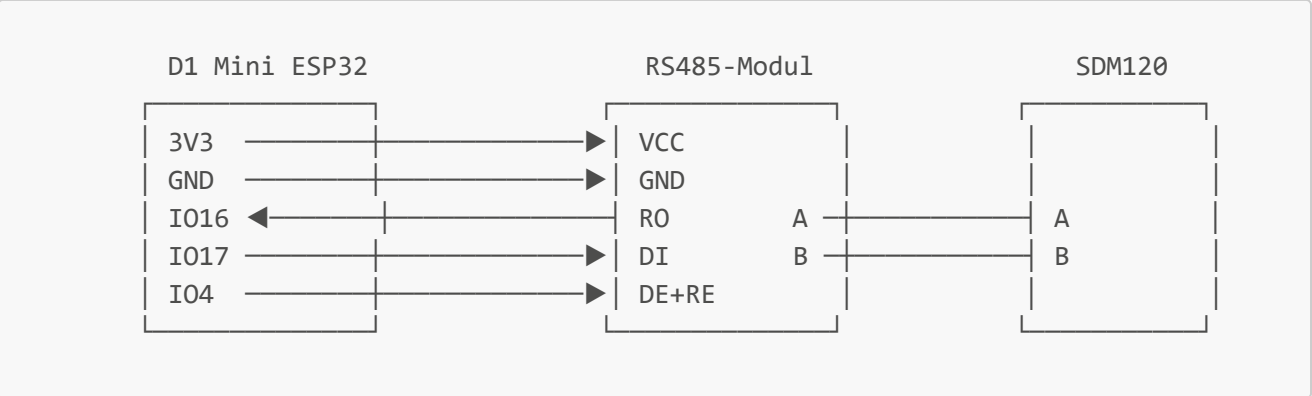
RS485-Modul	SDM120 Klemme
A (+)	A
B (−)	B

Hinweise zur Busleitung:

- Verwende ein **verdrilltes Adernpaar** für A und B.
- Bei **wenigen Metern** ist kein Abschlusswiderstand nötig.

- Bei **längeren Leitungen (über ca. 10 m)** gehört an beide Busenden je ein 120-Ω-Widerstand zwischen A und B. Viele Module haben ihn bereits bestückt (R7 bzw. „120R").
- Kommt keine Antwort, tausche **A und B**. Die Beschriftung ist nicht bei allen Herstellern einheitlich.

Schaltbild (vereinfacht)



⚠ Sicherheit

Der SDM120 wird im **230-V-Netz** installiert. Vom Gateway werden **nur die Modbus-Klemmen** angeschlossen. Einbau und Anschluss im Zähler- oder Verteilerschrank gehören in die Hände einer **Elektrofachkraft**. Das Netzteil muss für den Einbauort geeignet sein, zum Beispiel ein Hutschienennetzteil.

Bluetooth-Reichweite: Ein Verteilerschrank aus Metall dämpft Funk stark. Plane den ESP32 so ein, dass er außerhalb des Schranks sitzt oder eine externe Antenne hat. Das gilt auch für WLAN.

---

4. Einstellungen am SDM120

Werkseinstellung:

Parameter	Wert
Baudrate	2400
Datenformat	8 Datenbits, keine Parität, 1 Stoppbit (8N1)
Modbus-Adresse	1

Baudrate und Adresse lassen sich über das Setup-Menü am Zähler ändern. Die Bedienung beschreibt die Anleitung des Zählers. Die Werte in `config.h` (`MODBUS_BAUD`, `MODBUS_SLAVE`) müssen **identisch** mit dem Zähler sein.

---

5. Software einrichten

5.1 Arduino IDE

1. **ESP32-Boardpaket** installieren. Unter *Datei → Voreinstellungen → Zusätzliche Boardverwalter-URLs* trägst du ein:

`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
Danach installierst du im Boardverwalter das Paket „esp32" von Espressif.

2. Board-Einstellungen unter *Werkzeuge*:

Einstellung	Wert
Board	**ESP32 Dev Module** (der D1 Mini ESP32 ist ein ESP32-WROOM-32)
Partition Scheme	**Huge APP (3MB No OTA/1MB SPIFFS)**
Upload Speed	921600 (bei Problemen 115200)

⚠ Mit der **Standard-Partition (1,2 MB)** passt das Programm nicht in den Speicher, weil Bluetooth und WLAN zusammen zu groß sind. Die IDE meldet dann „Sketch too big". „Huge APP" bietet 3 MB für das Programm. Weil die Firmware kein OTA nutzt, wird die zweite App-Partition nicht gebraucht.

3. Bibliotheken über den Bibliotheksverwalter installieren:
   - **ModbusMaster** von Doc Walker
   - **PubSubClient** von Nick O'Leary (ab Version 2.8)

5.2 Projektstruktur

```
SDM120_Zaehler/
├── SDM120_Zaehler.ino   Hauptprogramm
├── config.h             alle Einstellungen
├── README.md            diese Dokumentation
└── APP_INVENTOR.md      Anleitung für die Handy-App
```

Der Ordnername muss dem Namen der `.ino` entsprechen, sonst öffnet die Arduino IDE das Projekt nicht korrekt.

5.3 Konfiguration (`config.h`)

Konstante	Bedeutung	Standard
`BT_ENABLED`	1 = Bluetooth aktiv	1
`BT_NAME`	Name beim Koppeln am Handy	`SDM120-Zaehler`
`BT_INTERVAL_MS`	Sendeintervall der Datenzeile	1000
`WIFI_ENABLED`	0 = reiner Bluetooth-Betrieb, ohne MQTT und Web	1
`WIFI_SSID`, `WIFI_PASS`	WLAN-Zugangsdaten	–
`HOSTNAME`	Gerätename, auch für mDNS	`sdm120`
`MQTT_ENABLED`	1 = MQTT aktiv	1
`MQTT_HOST`, `MQTT_PORT`	Broker	`192.168.1.10`, 1883

Konstante	Bedeutung	Standard
`MQTT_USER`, `MQTT_PASS`	leer = ohne Anmeldung	leer
`MQTT_BASE`	Basis-Topic	`strom/sdm120`
`MODBUS_BAUD`	wie am Zähler eingestellt	2400
`MODBUS_SLAVE`	Modbus-Adresse des Zählers	1
`PIN_RS485_RX/TX/DE`	Pinbelegung, DE = -1 für Auto-Modul	16 / 17 / 4
`POLL_INTERVAL_MS`	Abstand zwischen Messzyklen	2000
`MODBUS_PAUSE_MS`	Pause zwischen Telegrammen	30
`MAX_FEHLER_IN_FOLGE`	Fehlversuche, bevor die Werte als ungültig gelten	3

5.4 Flashen per USB

1. D1 Mini ESP32 per **USB-Datenkabel** an den PC anschließen. Ein reines Ladekabel reicht nicht.
2. Unter *Werkzeuge → Port* den COM-Port wählen. Erscheint keiner, fehlt der USB-Treiber: meist **CH340** oder **CP2102**, je nach Board.
3. *Hochladen* klicken. Bleibt die IDE bei „Connecting…" hängen, die **BOOT-Taste** gedrückt halten, bis der Upload startet.
4. Danach den Serial Monitor mit 115200 Baud öffnen und die Startmeldungen prüfen.

Für jedes Update ist das USB-Kabel nötig. Sitzt der ESP später fest eingebaut, lohnt sich ein von außen erreichbares USB-Verlängerungskabel.

---

6. Registertabelle SDM120 (verwendet)

Alle Werte sind **Input-Register (Modbus-Funktion 0x04)**. Jeder Wert belegt **2 Register**, also 4 Byte, als **IEEE-754-Float32, Big Endian** mit dem High-Word zuerst.

Register (hex)	Messgröße	Einheit	MQTT/JSON-Name	Position im BT-Protokoll
0x0000	Spannung	V	`spannung`	3
0x0006	Strom	A	`strom`	4
0x000C	Wirkleistung (negativ = Einspeisung)	W	`wirkleistung`	5
0x0012	Scheinleistung	VA	`scheinleistung`	6
0x0018	Blindleistung	var	`blindleistung`	7
0x001E	Leistungsfaktor	–	`leistungsfaktor`	8
0x0046	Netzfrequenz	Hz	`frequenz`	9

Register (hex)	Messgröße	Einheit	MQTT/JSON-Name	Position im BT-Protokoll
0x0048	Wirkenergie Bezug (Import)	kWh	`import_kwh`	10
0x004A	Wirkenergie Einspeisung (Export)	kWh	`export_kwh`	11
0x0156	Wirkenergie gesamt	kWh	`gesamt_kwh`	12

Weitere Register ergänzt du in der Tabelle `REGISTER[]` und mit einem passenden Feld in `struct Messwerte`. MQTT, JSON, Serial-Ausgabe und Bluetooth übernehmen den neuen Wert automatisch.

⚠ Ein neuer Eintrag in `REGISTER[]` verändert die **Länge der Bluetooth-Zeile**. Die App prüft auf genau 12 Felder (`APP_INVENTOR.md`, Abschnitt 5.8) und muss dann angepasst werden. Neue Register hängst du am besten **am Ende** der Tabelle an, dann bleiben die bisherigen Positionen gleich.

---

7. Schnittstellen

7.1 Bluetooth Classic (SPP)

- Gerätename: `SDM120-Zaehler` (`BT_NAME`)
- Nur **eine** Verbindung gleichzeitig. Das ist eine Eigenschaft von Bluetooth SPP.
- Zeilenbasiertes Textprotokoll, Trennzeichen `;`, Zeilenende `\n`, Dezimalpunkt `.`

Datenzeile (ESP → Handy, jede Sekunde, solange verbunden):

```
$SDM;<gueltig>;<U>;<I>;<P>;<S>;<Q>;<PF>;<f>;<Import>;<Export>;<Gesamt>
$SDM;1;231.4;1.23;245.7;284.6;-143.2;0.863;50.01;1234.567;0.000;1234.567
```

- `gueltig` ist `1`, wenn die letzte Messung ok war. `0` bedeutet, dass der Zähler `MAX_FEHLER_IN_FOLGE`-mal nicht geantwortet hat. Die Zahlen sind dann die zuletzt gültigen Werte.
- Ein Wert, der noch nie gelesen wurde, steht als `-` in der Zeile.

Befehle (Handy → ESP), jeweils mit `\n` abschließen:

Befehl	Antwort
`I`	`$INF;<fw>;<ip>;<rssi>;<ok>;<fehler>;<letzter_fehler>;<uptime_s>`

Weitere Befehle kommen in `btBefehlAusfuehren()` hinzu.

7.2 MQTT

Topic	Inhalt	Retained
`strom/sdm120/status`	`online` / `offline` (Last Will)	ja

Topic	Inhalt	Retained
`strom/sdm120/spannung`	z. B. `231.4`	ja
`strom/sdm120/strom`	z. B. `1.23`	ja
`strom/sdm120/wirkleistung`	z. B. `245.7`	ja
`strom/sdm120/scheinleistung`	…	ja
`strom/sdm120/blindleistung`	…	ja
`strom/sdm120/leistungsfaktor`	…	ja
`strom/sdm120/frequenz`	…	ja
`strom/sdm120/import_kwh`	…	ja
`strom/sdm120/export_kwh`	…	ja
`strom/sdm120/gesamt_kwh`	z. B. `1234.567`	ja
`strom/sdm120/json`	alle Werte + Status als JSON	nein

Veröffentlicht wird **nur nach einem vollständigen, plausiblen Messzyklus**, standardmäßig alle 2 s.

7.3 JSON-API (`GET /api`)

```
{
  "gueltig": true, "alter_s": 1,
  "spannung": 231.4, "strom": 1.23, "wirkleistung": 245.7,
  "scheinleistung": 284.6, "blindleistung": -143.2, "leistungsfaktor": 0.863,
  "frequenz": 50.01, "import_kwh": 1234.567, "export_kwh": 0.000, "gesamt_kwh":
1234.567,
  "ok": 812, "fehler": 3, "letzter_fehler": 226, "rssi": -61, "uptime_s": 4071
}
```

Feld	Bedeutung
`gueltig`	`false`, wenn der Zähler `MAX_FEHLER_IN_FOLGE`-mal nicht geantwortet hat
`alter_s`	Alter der letzten gültigen Messung in Sekunden
`ok` / `fehler`	Anzahl erfolgreicher bzw. fehlgeschlagener Zyklen seit dem Start
`letzter_fehler`	letzter ModbusMaster-Fehlercode, dezimal (siehe Kapitel 9)

Die API sendet den Header `Access-Control-Allow-Origin: *`.

7.4 Weboberfläche

`http://sdm120.local` (oder die IP-Adresse) zeigt Zählerstand, Leistung, alle Messwerte und den Status. Die Seite aktualisiert sich alle 2 s.

8. Funktionsweise der Firmware

8.1 Ablauf in `loop()`

```
loop()
 ├─ web.handleClient()    Weboberfläche / API          (nur WIFI_ENABLED)
 ├─ mqttTask()            Verbindung halten, Reconnect (nur MQTT_ENABLED)
 ├─ btTask()              Befehle lesen, Datenzeile senden (nur BT_ENABLED)
 └─ modbusTask()          pro Durchlauf genau EIN Modbus-Telegramm
```

8.2 Modbus-Sequenzer

Ein Messzyklus liest die 10 Werte aus `REGISTER[]` nacheinander. Pro `loop()`-Durchlauf wird **ein** Telegramm gesendet, dazwischen liegt eine Pause von `MODBUS_PAUSE_MS`. Bei 2400 Baud dauert ein Telegramm samt Antwort rund 80–100 ms, ein kompletter Zyklus also etwa 1,2 s. Während ModbusMaster auf eine Antwort wartet, ruft es über `sdm.idle()` regelmäßig `yield()` auf.

Antwortet der Zähler nicht, **bricht der Zyklus sofort ab**. Die Bibliothek wartet pro Anfrage fest 2 s. Ohne den Abbruch wären es bei einer Störung zehnmal 2 s, in denen auch Bluetooth und Web blockiert wären.

8.3 Plausibilitätsprüfung

Ein Zyklus wird nur übernommen, wenn

- die Spannung zwischen 0 und 300 V liegt,
- die Frequenz zwischen 45 und 65 Hz liegt,
- der Gesamtzählerstand nicht negativ ist und **nicht kleiner** als der zuletzt gültige Wert (Toleranz 0,01 kWh).

8.4 Float-Umwandlung

Der SDM120 liefert pro Wert zwei 16-Bit-Register. Die Firmware setzt sie zu einem 32-Bit-Wort zusammen (`high << 16 | low`) und kopiert die Bits mit `memcpy` in eine `float`-Variable.

8.5 Bluetooth und WLAN gleichzeitig

Der ESP32 teilt sich eine Funkeinheit für beides und schaltet im Zeitmultiplex zwischen Bluetooth und WLAN um. Dafür muss der **WLAN-Modem-Sleep aktiv bleiben**, was der Standard ist. Rufe also **nicht** `WiFi.setSleep(false)` auf, sonst stürzt der ESP beim Bluetooth-Start ab. Für die geringen Datenmengen hier ist der Durchsatz mehr als ausreichend.

---

9. Fehlersuche

Modbus-Fehlercodes (`letzter_fehler`, Info-Knopf der App, Serial-Ausgabe)

Code (hex)	dezimal	Bedeutung	typische Ursache
0xE2	226	Antwort-Timeout	A/B vertauscht, falsche Baudrate oder Adresse, DE/RE nicht angeschlossen, keine Versorgung am Modul
0xE3	227	CRC-Fehler	Störungen, fehlender Abschluss, unverdrillte Leitung, falsche Parität
0xE0	224	falsche Slave-ID in der Antwort	zweites Gerät mit gleicher Adresse am Bus
0xE1	225	falsche Funktion in der Antwort	gestörtes Telegramm
0x02	2	ungültige Registeradresse	Registertabelle passt nicht zum Zählermodell
0x01	1	ungültige Funktion	z. B. Holding- statt Input-Register gelesen

Checkliste bei „Zähler antwortet nicht"

1. Leuchtet die Power-LED am RS485-Modul?
2. Stimmen **Baudrate und Adresse** in `config.h` mit dem Zählermenü überein?
3. **A und B tauschen.**
4. Ist DE/RE an IO4 angeschlossen, oder bei einem Auto-Modul `PIN_RS485_DE = -1` gesetzt?
5. Sind RX und TX richtig herum? RO gehört an IO16, DI an IO17.
6. Serial Monitor mit 115200 Baud öffnen und die Meldungen `[MODBUS]` beobachten.

Bluetooth-Probleme

Symptom	Ursache / Lösung
Fehlermeldung beim Kompilieren „Bluetooth Classic (SPP) nicht verfuegbar"	Falsches Board gewählt (S3/C3). **ESP32 Dev Module** wählen.
„Sketch too big"	Partition Scheme auf **Huge APP (3MB No OTA/1MB SPIFFS)** stellen.
Handy findet „SDM120-Zaehler" nicht	Serial Monitor prüfen, ob `[BT] bereit` erscheint. `BT_ENABLED` muss 1 sein.
Verbindung klappt nicht	Ein anderes Gerät ist schon verbunden (nur eine Verbindung möglich). Kopplung am Handy löschen und neu koppeln.
ESP startet nach Bluetooth-Start neu	Kein `WiFi.setSleep(false)` verwenden (8.5). Netzteil zu schwach (mindestens 500 mA).

App-seitige Fehlersuche: siehe `APP_INVENTOR.md`, Kapitel 7.

---

10. Inbetriebnahme-Checkliste

- ☐ SDM120 von der Elektrofachkraft installiert, Anzeige läuft
- ☐ Baudrate und Adresse am Zähler abgelesen und in `config.h` eingetragen
- ☐ WLAN- und MQTT-Daten in `config.h` eingetragen (oder `WIFI_ENABLED 0`)
- ☐ Board **ESP32 Dev Module**, Partition **Huge APP (3MB No OTA/1MB SPIFFS)**
- ☐ RS485-Modul mit 3,3 V versorgt, Verdrahtung nach Kapitel 3
- ☐ Firmware per USB geflasht
- ☐ Serial Monitor zeigt `[BT] bereit`, die IP-Adresse und Zeilen `[SDM120] U=... E=... kWh`
- ☐ Zählerstand im Serial Monitor stimmt mit dem Display des SDM120 überein
- ☐ Handy mit „SDM120-Zaehler" gekoppelt
- ☐ App zeigt die Werte live, der Info-Knopf liefert die `$INF`-Zeile
- ☐ Weboberfläche `http://sdm120.local` erreichbar
- ☐ MQTT-Topics im Broker sichtbar (z. B. mit MQTT Explorer)

---

11. Erweiterungsideen

- **Schalten vom Handy:** Bluetooth-Befehle für Relais oder Shelly-Geräte über MQTT ergänzen, zum Beispiel bei Einspeiseüberschuss.
- **Historie:** Mosquitto → Telegraf / Node-RED → InfluxDB → Grafana
- **Tages- und Monatsverbrauch:** In der App mit TinyDB oder auf dem ESP mit NTP und Preferences speichern.
- **Browser-Visualisierung mit TMS Web Core:** MQTT over WebSockets. Bei Hosting auf GitHub Pages (HTTPS) muss der Broker per **WSS** erreichbar sein.
- **Mehrere Zähler:** RS485 ist ein Bus. Jeder Zähler bekommt eine eigene Adresse, und die Firmware fragt die Adressen nacheinander ab.

---

12. Versionshistorie

Version	Datum	Änderung
1.0	24.10.2026	Erste Version für D1 Mini (ESP8266): Modbus-Sequenzer, MQTT, Web/API, OTA
2.0	24.10.2026	Umstieg auf D1 Mini ESP32: Hardware-UART2 statt SoftwareSerial, Bluetooth Classic mit Zeilenprotokoll für App Inventor, WLAN abschaltbar, Messzyklus 2 s
2.1	24.10.2026	OTA entfernt, Programmieren nur per USB, Partition „Huge APP"